INFORME TÉCNICO DEL PROYECTO

Tucumán Respira

Panel web de monitoreo de calidad del aire IoT y firmwares de la estación (Arduino Uno + ESP32)

Encargado del trabajo: Benicio Gael Pieroni

Aspecto	Detalle
Encargado	Benicio Gael Pieroni
Proyecto	Tucumán Respira — Estación de monitoreo ambiental IoT
Entregables	Aplicación web estática (index.html · styles.css · app.js) y dos firmwares: Arduino Uno (sketch_oct17a) y ESP32 (sketch_oct17b)
Fuente de datos	Supabase (PostgreSQL) — tabla «mediciones» (API PostgREST)
Alcance de este informe	Documento único que consolida el informe del panel web, el detalle de los cambios de umbrales realistas y los firmwares con su integración y el control del zumbador.
Fecha del informe	18/10/2026
Estado	Implementado, integrado y verificado (pendiente de publicación online)

Este documento reemplaza y unifica los informes anteriores («Informe técnico del panel web» y «Cambios de umbrales realistas») y agrega los firmwares del Arduino Uno y del ESP32, la integración entre ambos, la base de datos y el control remoto del zumbador desde la web. Describe el diseño, la arquitectura, las funcionalidades, los umbrales de referencia, las correcciones aplicadas y las verificaciones realizadas durante el desarrollo del sistema «Tucumán Respira».

Índice de contenidos

1. Resumen ejecutivo
2. Contexto y objetivos del proyecto
3. Decisiones de diseño acordadas
4. Arquitectura y stack tecnológico
5. Estructura de archivos y métricas de código
6. Fuente de datos: Supabase
7. Métricas y umbrales de referencia
8. Cambios de umbrales realistas (antes / después)
9. Índice de Calidad del Aire (ICA / EPA)
10. Funcionalidades del panel web
11. Estados del sistema y comportamiento «offline»
12. Adaptación responsive (móviles y tablets)
13. Seguridad
14. Firmware del Arduino Uno (adquisición y GPS)
15. Firmware del ESP32 (WiFi, Supabase y alarma)
16. Integración del sistema y control del zumbador
17. Correcciones de errores durante el desarrollo
18. Verificaciones y pruebas realizadas
19. Cómo ejecutar y visualizar el sistema
20. Pendientes y recomendaciones

Anexo A — Configuración y claves

Anexo B — Exportación de datos (CSV / JSON)

Anexo C — Trama de comunicación Uno → ESP32

Anexo D — Prompt para recrear el proyecto con opencode

1. Resumen ejecutivo

«Tucumán Respira» es una estación de monitoreo ambiental que mide la calidad del aire y variables meteorológicas y publica sus lecturas en la nube. El sistema se compone de cuatro piezas: un **Arduino Uno** que adquiere los sensores y calcula la posición GPS; un **ESP32** que se conecta a Wi-Fi, envía las mediciones a **Supabase** y gobierna una alarma sonora con zumbador; una **base de datos PostgreSQL** en Supabase; y una **aplicación web estática** que rediseñada por completo muestra los datos en vivo y permite descargarlos y detener la alarma de forma remota.

Este informe único documenta el panel web, los umbrales realistas, los dos firmwares y su integración. Entre los resultados se destacan: un panel claro con datos en tiempo real y respaldo por sondeo, 8 métricas con umbrales por estándar (EPA, OMS/ICNIRP, OSHA/NIOSH y ASHRAE) con su fuente visible, el ICA (EPA) con PM2.5 y CO, histórico y gráficos, descarga pública en CSV y JSON, mapa con OpenStreetMap (sin clave de API) que conserva la última coordenada GPS válida, y un botón «Detener zumbador» que descubre solo la dirección IP de la estación desde la base de datos.

Resultados principales

• Panel web estático (HTML + CSS + JS) con datos en vivo desde Supabase y respaldo por sondeo cada 15 s.

• 8 métricas con umbrales por estándar, cada una con su fuente visible, todas con nivel de peligro.

• Índice de calidad del aire (ICA/EPA) calculado con PM2.5 y CO.

• Histórico por sensor, gráfico de tendencia y descarga pública en CSV y JSON (100, 500, 1000 o todas las filas).

• Mapa de ubicación con OpenStreetMap y marcador de la estación; usa la última coordenada GPS válida.

• Botón «Detener zumbador» en la web: la IP de la estación se obtiene sola desde Supabase.

• Firmware del Arduino Uno: 8 sensores, GPS NMEA y trama de 9 campos hacia el ESP32.

• Firmware del ESP32: Wi-Fi (modo estación + punto de acceso de configuración), carga a Supabase, alarma por zumbador y endpoints de control (/status, /alarma, /silenciar).

• Diseño responsive con puntos de quiebre en 1024, 640 y 400 píxeles, sin scroll horizontal en celulares.

Estado actual verificado en la base de datos al 18/10/2026: **1.284 lecturas**, la última el 18/10/2026 a las 15:50:53 UTC. De ellas, 533 incluyen coordenadas GPS y 751 son anteriores al soporte de GPS (sin latitud/longitud). La comunicación en vivo funciona (última lectura hace minutos).

2. Contexto y objetivos del proyecto

«Tucumán Respira» es una estación de monitoreo ambiental de bajo costo construida con microcontroladores (Arduino Uno R3 y ESP32) y sensores de calidad del aire y clima. Las mediciones se envían de forma periódica a Supabase y quedan disponibles para consulta pública.

Objetivo general

Ofrecer un sistema completo de monitoreo —desde la adquisición en campo hasta la publicación web— que permita al público comprender el estado de la calidad del aire y consultar los datos históricos.

Objetivos específicos

• Medir en forma continua variables de calidad del aire (PM2.5, CO■, CO), meteorológicas (temperatura, humedad, presión), radiación UV y luz ambiental.

• Registrar la ubicación geográfica (GPS) de las mediciones.

• Publicar los datos en la nube y ofrecer un panel público que los muestre de forma clara.

• Avisar localmente (alarma sonora) cuando la calidad del aire supera umbrales peligrosos, y permitir silenciar esa alarma desde la aplicación web.

• Permitir la descarga libre de los datos crudos (CSV/JSON).

• Garantizar una correcta visualización en celulares y aplicar correcciones de seguridad.

Perfil de usuarios

Público general: docentes, estudiantes y personas técnicas o interesadas en la calidad del aire. La interfaz prioriza la legibilidad, el lenguaje claro y la accesibilidad desde cualquier dispositivo.

3. Decisiones de diseño acordadas

Las siguientes decisiones fueron acordadas con el responsable del proyecto antes y durante la implementación, y definen el comportamiento actual del sistema.

Tema	Decisión adoptada
Base de partida	Versión simple existente del panel (se rediseñó sobre ella).
Tipo de proyecto	Proyecto real, conectado a la base de datos real de la estación.
Datos simulados	No se muestran datos simulados. Solo se presentan mediciones reales.
Formato de entrega	Aplicación estática de 3 archivos (HTML + CSS + JS) más dos firmwares.
Identidad visual	Nombre «Tucumán Respira», tema oscuro/claro con verde esmeralda.
Público objetivo	General: docentes, estudiantes y técnicos.
Prioridad	Estado de la calidad del aire + valor de cada sensor.
Interacción	Clic en un sensor abre su histórico (panel modal).
Descarga	Botón público, formatos CSV y JSON, últimas 100 por defecto.
Alertas y peligro	Cada métrica tiene niveles de peligro con su fuente; el estado general (ICA) se calcula con PM2.5 y CO.
Fuente del umbral	Se muestra en la interfaz la fuente de cada umbral.

Tema	Decisión adoptada
Actualización	Tiempo real + marca de última actualización (fecha y hora + relativa).
Mapa	Ubicación GPS de la estación sobre mosaicos de OpenStreetMap (sin clave).
Zumbador	Ubicado y gobernado por el ESP32 (GPIO 23); la alarma la decide la propia ESP32.
GPS	Conectado al Arduino Uno en los pines 7 (RX) y 8 (TX) por SoftwareSerial.
Detener alarma	Botón directo en la app que consulta a la ESP32; la IP se obtiene sola desde Supabase.
Base de datos	No se cambian los campos existentes; solo se agrega «ip_dispositivo».
Control de versiones	No se modificó el repositorio Git durante el desarrollo.
Seguridad	Se aplicaron las correcciones de seguridad indicadas.

4. Arquitectura y stack tecnológico

La arquitectura es de cuatro capas con comunicación en cascada:

• **Adquisición (Arduino Uno):** lee los sensores y el GPS y envía una trama de 9 campos por puerto serie al ESP32.

• **Enlace y nube (ESP32):** se conecta al Wi-Fi del lugar, expone un punto de acceso de configuración y realiza peticiones HTTPS a la API REST de Supabase (PostgREST).

• **Persistencia (Supabase):** base PostgreSQL con la tabla «mediciones».

• **Presentación (app web):** aplicación estática que lee Supabase y, para la alarma, consulta directamente al ESP32 en la red local.

Flujo de datos: Estación (Arduino Uno → ESP32) → Supabase (PostgreSQL + Realtime) → navegador (fetch REST + canal Realtime) → render de la interfaz.

Tecnologías y librerías utilizadas

Recurso	Versión / origen	Uso en el proyecto
HTML5 / CSS3 / JavaScript	Nativo	Estructura, estilos y lógica de la aplicación.
Lucide	unpkg @latest (v1.56.0)	Iconografía de la interfaz (SVG).
Chart.js	CDN 4.4.1	Gráfico de tendencia y gráficos de histórico por sensor.
Leaflet	CDN 1.9.4	Mapa de ubicación de la estación.
supabase-js	CDN @supabase/supabase-js@2	Cliente Realtime (suscripción a nuevas filas).
Supabase REST	API PostgREST	Lectura de mediciones y descargas.
Arduino Uno	ATmega328P (core avr 1.8.8)	Lectura de sensores y GPS; trama serie.
ESP32	ESP32 (core 3.3.12)	Wi-Fi + HTTPS, alarma y portal de configuración.
Supabase	PostgreSQL / PostgREST	Almacenamiento y API REST.

Justificación de las librerías

Se optó por librerías livianas y ampliamente utilizadas, cargadas por CDN para mantener la aplicación en solo 3 archivos. Se descartó el uso de frameworks CSS pesados (por ejemplo, Tailwind) y de Font Awesome, reemplazando los íconos por Lucide (SVG) y los estilos por CSS propio, con el objetivo de reducir el peso y mantener el control total del diseño. Para el mapa se pasó de Carto a OpenStreetMap para no depender de una clave de API.

5. Estructura de archivos y métricas de código

El proyecto se organiza en la aplicación web («App-Web2») y los firmwares («Firmwares_Tucuman_Respira»).

Archivo	Rol	Líneas aprox.
index.html	Estructura y semántica de la interfaz; carga de librerías y de app.js.	271
styles.css	Sistema de diseño: tokens de tema, componentes y reglas responsive.	678

Archivo	Rol	Líneas aprox.
app.js	Configuración, datos, clasificación, gráficos, mapa, modales y descargas.	1012
sketch_oct17a.ino	Firmware del Arduino Uno (adquisición, GPS y trama serie).	274
sketch_oct17b.ino	Firmware del ESP32 (WiFi, Supabase, alarma y portal).	842
Total	—	≈ 3.077

Organización interna de app.js

• CONFIG: parámetros centrales (URL y clave de Supabase, tabla, límites, tiempos, coordenadas por defecto, clave del dispositivo y minutos de silencio).

• METRICS: definición de los 8 sensores, unidades, decimales, umbrales y fuentes.

• TREND_TABS y AQI_*: pestañas del gráfico de tendencia y tablas del índice de calidad del aire.

• Utilidades: formato de fecha/hora, tiempo relativo, formateo, escapado de CSV y validación de GPS.

• ICA (EPA): cálculo del subíndice y del índice global.

• Render: construcción de tarjetas, banner, resumen, reloj de estado, mapa, gráficos, modales y descargas.

• Carga de datos: lectura inicial, Realtime y sondeo de respaldo; e inicialización.

6. Fuente de datos: Supabase

La información proviene de un proyecto de Supabase (PostgreSQL) con la API PostgREST habilitada. La tabla utilizada es «mediciones».

Campo	Tipo	Significado	Unidad
id	entero	Identificador autoincremental de la fila.	—
created_at	timestamp	Fecha y hora de la lectura (UTC).	ISO-8601
co2	numérico	Dióxido de carbono.	ppm
co	numérico	Monóxido de carbono.	ppm
pm25	numérico	Material particulado fino (PM2.5).	µg/m³
temperatura	numérico	Temperatura ambiente.	°C
humedad	numérico	Humedad relativa.	%
presion	numérico	Presión atmosférica.	hPa
uv	numérico	Índice de radiación ultravioleta.	índice
luz	numérico	Luminosidad ambiental.	lux
latitud	numérico	Latitud GPS de la estación.	grados
longitud	numérico	Longitud GPS de la estación.	grados
ip_dispositivo	texto	IP local actual del ESP32 (nueva columna).	IPv4

Estado verificado de la base de datos (al 18/10/2026)

Métrica	Valor
Filas totales	1.284
Primera lectura	05/10/2026 00:45:18 UTC
Última lectura	18/10/2026 15:50:53 UTC
Lecturas con coordenadas	533
Lecturas sin coordenadas (firmware previo sin GPS)	751
Lecturas con «ip_dispositivo»	117 (desde que se activó la publicación de la IP)

La columna «ip_dispositivo» permite que la aplicación descubra sola la dirección de la estación en la red local y, con ella, detener el zumbador sin que el usuario tenga que escribir ninguna IP.

7. Métricas y umbrales de referencia

Cada sensor se clasifica según rangos de referencia tomados de fuentes reconocidas de salud y seguridad. Todas las métricas cuentan con niveles que llegan a «peligro», cada uno con su fuente visible en la interfaz. El estado general (índice de calidad del aire) se calcula con PM2.5 y CO; el resto de las métricas marcan su propio nivel de alerta en su tarjeta.

Métrica	Unidad	Umbrales y etiquetas	Fuente
CO■	ppm	Bueno ≤800 · Aceptable ≤1000 · Ventilación pobre ≤1500 · Alta (PEL 8 h) ≤5000 · Muy alta (STEL) ≤30000 · Peligro >30000	OSHA/NIOSH · ASHRAE (ventilación)
CO (ICA)	ppm	Bueno ≤4,4 · Moderado ≤9,4 · Grupos sensibles ≤12,4 · Dañino ≤15,4 · Muy dañino ≤30,4 · Peligroso ≤50,4 · Extremo >50,4	EPA AQI (8 h) · OSHA
PM2.5 (ICA)	µg/m³	Bueno ≤9,0 · Moderado ≤35,4 · Grupos sensibles ≤55,4 · Dañino ≤125,4 · Muy dañino ≤225,4 · Peligroso >225,4	EPA AQI 2024 (24 h)
Temperatura	°C	Hipotermia ≤0 · Frío ≤10 · Confortable ≤27 · Precaución por calor ≤32 · Extrema precaución ≤39 · Peligro ≤52 · Peligro extremo >52	NWS/OSHA (calor) · NIOSH (frío)
Humedad	%	Muy seco ≤20 · Seco ≤30 · Ideal ≤60 · Algo húmedo ≤70 · Riesgo de moho ≤85 · Alto riesgo de moho >85	OMS (humedad y moho) · ASHRAE
Presión	hPa	Baja ≤1004,9 · Normal ≤1020 · Alta >1020	Informativo (sin umbral de salud)
UV	índice	Bajo ≤2 · Moderado ≤5 · Alto ≤7 · Muy alto ≤10 · Extremo >10	OMS / ICNIRP
Luz	lux	Oscuridad ≤9,9 · Penumbra ≤99,9 · Interior ≤499 · Pleno sol >499	Informativo (sin umbral de salud)

Notas

• Los valores son de referencia para interpretar las mediciones y no constituyen diagnóstico sanitario.

• El calor y el frío se evalúan por temperatura seca; el índice de calor real depende además de la humedad y el viento.

• La OMS no fija un umbral cuantitativo de humedad: los niveles altos indican riesgo de moho, no peligro agudo.

• Presión y luz se muestran como referencia: no existe un umbral de peligro para la salud humana en esos rangos.

• El CO■ alto es peligroso solo en niveles ocupacionales (OSHA/NIOSH); en interiores es ante todo un indicador de ventilación.

• El estado general (ICA) se calcula con PM2.5 y CO; cada métrica, además, marca su propia alerta en la tarjeta.

8. Cambios de umbrales realistas (antes / después)

Como parte del proyecto se revisaron y recalibraron los umbrales de todas las métricas contra fuentes de salud y seguridad pública, se corrigieron las etiquetas que sugerían una norma inexistente y se fijó la fuente visible de cada umbral. El objetivo fue que cada sensor clasifique con niveles de peligro **realistas** y honestos, sin rangos inventados.

3.1 Valores que cambiaron

Métrica	Antes	Ahora	Fuente
CO■	Bueno ≤800 · Aceptable ≤1000 · Ventilar ≤2000 · Muy alto ≤5000 · Riesgo >5000 (fuente «ASHRAE»).	Bueno ≤800 · Aceptable ≤1000 · Ventilación pobre ≤1500 · Alta (PEL 8 h) ≤5000 · Muy alta (STEL) ≤30000 · Peligro >30000.	OSHA/NIOSH · ASHRAE
Temperatura	Frío ≤16,9 · Fresco ≤19,9 · Confortable ≤26 · Cálido ≤29,9 · Extremo >29,9 (fuente «ASHRAE 55»).	Hipotermia ≤0 · Frío ≤10 · Confortable ≤27 · Precaución por calor ≤32 · Extrema precaución ≤39 · Peligro ≤52 · Peligro extremo >52.	NWS/OSHA (calor) · NIOSH (frío)
Humedad	Seco ≤29,9 · Ideal ≤60 · Húmedo >60 (fuente «ASHRAE 62.1»).	Muy seco ≤20 · Seco ≤30 · Ideal ≤60 · Algo húmedo ≤70 · Riesgo de moho ≤85 · Alto riesgo de moho >85.	OMS (humedad y moho) · ASHRAE

Sin cambios (ya eran correctos frente a su norma): PM2.5 (EPA AQI 2024), CO (EPA AQI 8 h / OSHA) y UV (OMS/ICNIRP). Se activó el resaltado de alerta para CO■, temperatura y humedad, de modo que cuando alcanzan un nivel marcado en rojo su tarjeta se destaca. Presión y luz quedaron como referencia, porque no tienen umbral de peligro real.

3.2 Texto del panel «Acerca del sistema» (index.html)

El texto anterior decía que solo PM2.5, CO y UV generaban alertas y mencionaba el UV como parte del estado general, lo que no era exacto. Se reemplazó por:

```
El estado general (indice de calidad del aire) se calcula con PM2.5 y CO segun el metodo de la
EPA. Los demas parametros usan umbrales propios basados en fuentes de salud publica
(OSHA/NIOSH, OMS y NWS) para indicar su nivel; cuando alcanzan un valor danino, su tarjeta se
resalta como alerta.
```

3.3 Criterio de umbrales y fuentes

Métrica	Fuente y criterio
PM2.5 (ICA)	EPA AQI 2024. Valores oficiales de subíndice (24 h).
CO (ICA)	EPA AQI (8 h) y OSHA. Valores oficiales de subíndice.
UV	OMS / ICNIRP (Guía Solar UV). Bajo, Moderado, Alto, Muy alto, Extremo.
CO■	OSHA/NIOSH (PEL 5000 ppm 8 h; STEL 30000 ppm; IDLH 40000 ppm) y ASHRAE como referencia de ventilación.
Temperatura	NWS/OSHA (estrés por calor) y NIOSH (estrés por frío), sobre temperatura seca.
Humedad	OMS (humedad y moho) y ASHRAE (recomendación 30–60 %). Los niveles altos indican riesgo de moho.
Presión / Luz	Referencia. No existe un umbral de peligro para la salud en esos rangos.

3.4 Aclaraciones honestas

• El calor real depende también de la humedad y el viento (índice de calor); aquí se evalúa por temperatura seca.

• La OMS no fija un umbral cuantitativo de humedad; por eso los valores altos se rotulan como «riesgo de moho».

• El CO■ solo es tóxico en concentraciones ocupacionales; en interiores es, sobre todo, un indicador de ventilación.

• El índice ICA de la EPA se basa en promedios (PM2.5 24 h, CO 8 h); la app usa la lectura del momento, tal como se acordó.

• Presión y luz se dejan como referencia para no inventar un peligro que la ciencia no respalda.

9. Índice de Calidad del Aire (ICA / EPA)

El estado general de la calidad del aire se resume con un índice estilo ICA (Air Quality Index de la EPA). Se calcula el subíndice de PM2.5 y el de CO y se toma el peor de los dos como índice global del momento. La alarma sonora del ESP32 replica este cálculo.

Rango ICA	Categoría	Descripción
0 – 50	Buena	La calidad del aire es satisfactoria y el riesgo para la salud es bajo.
51 – 100	Moderada	Calidad aceptable; personas muy sensibles podrían notar molestias leves.
101 – 150	Dañina para grupos sensibles	Los grupos sensibles pueden experimentar efectos en la salud.
151 – 200	Dañina	Todos pueden comenzar a sentir efectos en la salud.
201 – 300	Muy dañina	Riesgo para la salud de toda la población.
301 – 500	Peligrosa	Condiciones de emergencia sanitaria.

Además del número y la categoría, el panel muestra la descripción, una recomendación sanitaria acorde y un indicador del estado de la alerta del equipo (NORMAL, PRECAUCIÓN, ALERTA o ALERTA CRÍTICA).

Interpolación lineal

El subíndice se obtiene por interpolación lineal dentro del rango correspondiente (por ejemplo, para PM2.5 con concentración 12 µg/m³ corresponde al tramo 9,1–35,4 → subíndice 51–100). El método replica el empleado por la EPA. La alarma sonora se activa cuando el ICA supera 100 y se corta automáticamente cuando baja de 80.

10. Funcionalidades del panel web

10.1 Encabezado y estado de conexión

• Barra superior fija («sticky») con la identidad «Tucumán Respira»; el logo recarga la página.

• Insignia de conexión con indicador de color: en vivo, retraso o sin conexión, con tiempo relativo.

• Marca de última actualización con fecha y hora completas (dd/mm/aaaa hh:mm:ss).

• Botones de acción: descargar, cambiar tema, información y enlace a GitHub.

10.2 Banner de calidad del aire y tarjetas de sensores

• Número ICA grande, categoría, descripción, recomendación sanitaria e indicador de la alerta del equipo.

• Ocho tarjetas (CO■, CO, PM2.5, temperatura, humedad, presión, UV y luz) con valor, unidad, etiqueta de clasificación y barra de nivel; se resaltan en nivel peligroso.

• Al tocar una tarjeta se abre su histórico.

10.3 Resumen reciente y gráfico de tendencia

• Mín./máx. de temperatura, pico de PM2.5, promedio de CO■ y de humedad de las últimas 100 lecturas.

• Pestañas Aire (PM2.5 y CO), Clima (temperatura y humedad), Gases (CO■ y CO) y Luz/UV.

10.4 Histórico por sensor (modal)

• Estadísticas (actual, mín., promedio, máx.), selector de 20/50/100 puntos y gráfico del sensor.

• Se construye con el modal ya abierto y se redimensiona para evitar gráficos colapsados.

10.5 Mapa de ubicación

• Mapa construido con **Leaflet** y mosaicos de **OpenStreetMap** (sin clave de API), con marcador de la estación.

• Muestra la insignia «Ubicación por GPS» y las coordenadas; si la lectura más reciente no tiene posición válida, se usa la **última coordenada GPS válida** y se indica «Última ubicación conocida».

• Incluye un enlace «Ver en Google Maps» cuando hay coordenadas y recalcula su tamaño al girar el dispositivo.

10.6 Descarga pública de datos

• Formatos CSV (planilla) y JSON (datos crudos); rangos: últimas 100 (por defecto), 500, 1000 o todas.

• La descarga de «todas» usa paginación por rangos para traer el conjunto completo.

10.6b Control del zumbador desde la app

• Botón «**Detener zumbador**» en el panel de alerta.

• La aplicación obtiene la IP de la estación **automáticamente** desde el último registro de Supabase (campo «ip_dispositivo»); si aún no hay ninguno, usa 192.168.4.1 (punto de acceso de la ESP32).

• El botón envía la orden al endpoint /silenciar del ESP32; la respuesta indica los minutos de silencio.

10.6b Tema y detalles

• Tema claro/oscuro persistente en el navegador; gráficos y mapa se recolorean al cambiar.

• Aviso automático de estación desconectada, descartable, y actualización Realtime + sondeo cada 15 s.

11. Estados del sistema y comportamiento «offline»

La aplicación contempla y comunica con claridad los distintos estados de los datos, un aspecto clave cuando la estación no transmite.

Situación	Comportamiento de la interfaz
Cargando	Tarjetas «esqueleto» animadas; indicador «Conectando…».
En vivo (≤120 s)	Indicador verde con pulso; texto «En vivo · hace X».
Retraso (≤300 s)	Indicador ámbar; texto «Retraso · hace X».
Sin conexión (>300 s)	Indicador rojo; aviso con la fecha y hora de la última lectura.
Sin datos	Valores «--» y etiqueta «Sin datos»; ICA mostrado como «--».
Campo puntual nulo	Se muestra «--» solo en esa métrica, sin afectar al resto.
Sin GPS	Se usa la última coordenada válida; si no hay ninguna, el mapa se centra por defecto.

Un vigilante («watchdog») actualiza el estado cada segundo y recalcula el tiempo transcurrido respecto de la última lectura, de modo que el mensaje siempre refleja la realidad, incluso si no llegan datos nuevos.

12. Adaptación responsive (móviles y tablets)

Ancho	Cambios principales
≤ 1024 px	El banner y el resumen pasan a una columna; sensores en 2 columnas; tendencia y mapa se apilan.
≤ 640 px	Marca compacta; insignia de conexión a ancho completo; una columna de sensores; tipografías y paddings reducidos; gráficos y mapa de 240–260 px; campos de descarga en una columna; modales con menos margen.
≤ 400 px	Botones, tipografías y pestañas aún más compactos.

• El gráfico del histórico se construye con el modal ya abierto para evitar gráficos colapsados.

• El mapa recalcula su tamaño al girar el dispositivo o redimensionar la ventana.

• El aviso «offline» se envuelve en un contenedor con el ancho y los márgenes del resto; no desborda.

• El modal de descarga usa columnas minmax(0,1fr) y min-width:0 para no desbordar al abrir los desplegables en móviles, y un ancho que nunca supera la pantalla.

• Se bloquea el desplazamiento del fondo al abrir un modal y se desactivan animaciones si el usuario configuró «reducir movimiento».

13. Seguridad

• Se utiliza la clave pública (anon) de Supabase, diseñada para exponerse en el cliente, y no la clave de servicio («service_role»), que nunca debe incluirse.

• La aplicación es de solo lectura sobre los datos de la estación.

• La protección real de los datos depende de las políticas RLS (Row Level Security) configuradas en Supabase, que deben restringir el acceso a lectura pública y bloquear escritura/borrado.

• Se centralizó la configuración en el bloque CONFIG del inicio de app.js.

- El endpoint «/silenciar» del ESP32 exige una clave compartida («k») para evitar usos no autorizados.
- No se exponen datos sensibles del usuario; la descarga es pública por decisión del proyecto.

14. Firmware del Arduino Uno (adquisición y GPS)

El Arduino Uno actúa como **maestro de adquisición**. Lee los sensores, interpreta las sentencias NMEA del módulo GPS y envía periódicamente una trama de texto al ESP32 por puerto serie.

Sensores y pines

Sensor	Magnitud	Conexión
BME280 (GY-BMEP280)	Temperatura, humedad, presión	I²C (A4/A5), dirección 0x76/0x77
MH-Z19C	CO■ (ppm)	SoftwareSerial RX=10, TX=11
MQ-7	Monóxido de carbono (CO)	Analógico A0
Módulo LDR («MH-MQ»)	Luz ambiental (lux aproximado)	Analógico A1 · digital D12
GY-8511 (ML8511)	Radiación UV	Analógico A2
GY-GPS6MV2 (NEO-6M)	Latitud y longitud	SoftwareSerial RX=7, TX=8
ESP32	Enlace serie de datos	SoftwareSerial RX=2, TX=3

Funcionamiento

• El GPS se deja como receptor activo por defecto (gpsSerial.listen()); se conmuta brevemente al puerto del sensor de CO■ para leerlo y se vuelve al GPS.

• Se interpretan las sentencias **$GPGGA** y **$GNGGA**: se verifica la calidad del fix y se convierten latitud/longitud de formato NMEA (ggmm.mmmm) a grados decimales, aplicando el signo según N/S y E/W.

• El MQ-7 requiere un calentamiento inicial; el sistema espera 30 segundos antes de empezar.

• Las lecturas del BME280 se protegen contra valores inválidos (NaN).

• Cada 3 segundos se envía al ESP32 una trama de 9 campos.

Conservación de la última coordenada válida

Si el GPS pierde el fix momentáneamente, el firmware **no** envía (0,0): conserva la última posición válida y la reutiliza hasta recuperar la señal. Esto evita filas sin coordenadas originadas por cortes transitorios del GPS.

15. Firmware del ESP32 (WiFi, Supabase y alarma)

El ESP32 es el **enlace con la nube** y el **controlador de la alarma**. Recibe la trama del Arduino Uno, arma el registro y lo publica en Supabase; además, mide el ICA y gobierna el zumbador.

Conectividad

• Conexión al Wi-Fi del lugar en modo estación (STA), con reconexión automática.

• Punto de acceso permanente **Tucuman-Respira-Config** (contraseña Tucuman2026, IP 192.168.4.1) con portal DNS cautivo, para configurar la red sin flashear.

• Las credenciales se guardan en memoria no volátil (NVS) con Preferences.

Publicación en Supabase

• Petición HTTPS POST a la API REST de Supabase con encabezados apikey y Authorization (clave anon).

• Campos enviados: co2, co, pm25, temperatura, humedad, presion, uv, luz, latitud, longitud e **ip_dispositivo** (IP local del ESP32, para el descubrimiento automático desde la app).

• Se eliminó el envío de un campo inexistente («aqi») que provocaba un error HTTP 400.

• El sensor de partículas (PMS5003) se lee por Serial2 (RX 18 / TX 5, SET 4).

Alarma sonora

• Zumbador pasivo en GPIO 23, controlado con tone()/noTone().

• El ICA se calcula localmente con los mismos breakpoints EPA 2024 que la web.

• Se activa el sonido al superar ICA > 100 y se corta al bajar de 80; el silencio manual dura 10 minutos.

Endpoints HTTP

Ruta	Función
/silenciar?k=CLAVE	Silencia el zumbador durante 10 minutos (requiere la clave del dispositivo).
/status	Estado del dispositivo: conexión WiFi, ICA, alerta y silencio.
/alarma	Estado de la alarma (ICA, alerta, zumbando, silenciado).
/save (POST)	Guarda las credenciales WiFi y confirma si logró conectarse.
/ (portal)	Página de configuración; muestra el estado de la conexión WiFi.

Todos los endpoints de control envían encabezados CORS (Access-Control-Allow-Origin: *) para poder ser llamados desde la aplicación web.

16. Integración del sistema y control del zumbador

El flujo completo de una medición es:

• El Arduino Uno lee los sensores y el GPS y envía la trama de 9 campos al ESP32 por puerto serie.

• El ESP32 interpreta la trama, calcula el ICA y publica el registro en Supabase (incluida su IP).

• La aplicación web lee Supabase, muestra los datos y guarda la IP publicada por el ESP32.

• Al pulsar «Detener zumbador», la app llama a http://<ip>/silenciar?k=… y el ESP32 silencia el zumbador. No hace falta escribir ninguna IP: se toma sola desde la base de datos.

Requisitos de red

• El ESP32 y el dispositivo que usa la app deben estar en la misma red local para el control del zumbador.

• Si la estación está únicamente en modo punto de acceso, la IP del zumbador es siempre 192.168.4.1 (valor por defecto en la app).

Nota: la app se sirve por HTTP (por ejemplo con Live Server), por lo que la llamada al ESP32 por HTTP no presenta problemas de contenido mixto.

17. Correcciones de errores durante el desarrollo

Problema detectado	Causa	Solución aplicada
Los modales no se cerraban y aparecían encimados.	Reglas CSS .modal con display:grid/flex pisaban el atributo «hidden».	Se agregó [hidden]{display:none !important}.
El gráfico del histórico se veía colapsado en móvil.	Se creaba con el modal oculto (sin tamaño).	Se reordenó: abrir modal → crear gráfico → redimensionar.
El botón de GitHub aparecía vacío.	Las versiones nuevas de Lucide ya no incluyen íconos de marcas («github»).	Se incorporó el logotipo de GitHub como SVG propio en el botón.
El aviso «offline» desbordaba el borde en el celular.	Caja fuera del contenedor y sin márgenes.	Se envolvió el aviso en un .container, compartiendo el ancho del resto.
El logo del encabezado enlazaba a la raíz del sitio.	Era un enlace href a «./»; al abrir la app como archivo, el navegador mostraba el listado de la carpeta (exponía el directorio).	Se convirtió el logo en un botón que recarga la página (location.reload()).
Los paneles de Tendencia/GPS se salían de la tarjeta en móviles angostos.	El lienzo de Chart.js tiene ancho intrínseco de 300 px.	Grillas con minmax(0, …) y min-width:0 en .panel y .chart-wrap.
Umbrales sin respaldo normativo y con etiquetas inexactas.	Rangos elegidos a criterio propio.	Se recalibraron con fuentes (OSHA/NIOSH, EPA, OMS/ICNIRP, NWS, ASHRAE) y se corrigieron los textos.
El mapa mostraba «API key required» sobre fondo negro.	Los mosaicos de Carto requieren clave.	Se cambió a mosaicos de OpenStreetMap (sin clave).
La descarga JSON salía «entrecortada».	Pretty-print con muchos registros y MIME sin charset.	JSON sin indentación y MIME application/json;charset=utf-8.
El modal de descarga se desbordaba en Chrome Android al abrir los desplegables.	Grillas 1fr que no podían encogerse por el ancho intrínseco del <select>.	minmax(0,1fr), min-width:0, overflow-x:hidden y ancho del modal limitado al viewport.
El texto «JSON (datos crudos)» se cortaba en PC.	Modal demasiado angosto (420 px).	Se ensanchó el modal de descarga a 560 px.
La app enviaba datos a Supabase y recibía HTTP 400.	Se enviaba un campo «aqi» inexistente.	Se eliminó el campo del payload.
Las lecturas quedaban desalineadas (valores cruzados).	El Uno enviaba 8 campos y el ESP32 interpretaba 7.	Se unificó la trama en 9 campos y el parser correspondiente.
Llegaban coordenadas (0,0) a la base.	El GPS enviaba 0,0 al perder el fix.	El Uno conserva y reenvía la última coordenada válida.
El sketch del Arduino Uno quedó sobrescrito con el de la ESP32.	Reemplazo accidental del archivo.	Se restauró el firmware del Uno y se separaron ambos en carpetas propias.

18. Verificaciones y pruebas realizadas

Verificación	Resultado
Sintaxis del JavaScript (análisis con esprima).	Correcta (sin errores de análisis).
Coincidencia de los identificadores usados por app.js con el HTML.	Todos presentes.
Respuesta de la API REST de Supabase.	HTTP 200; columnas con los nombres esperados.
Escritura real a Supabase (POST desde el flujo).	HTTP 201 (registro creado correctamente).
Compilación del firmware del Arduino Uno (avr-g++).	Sin errores.
Compilación del firmware del ESP32 (xtensa-esp32-elf-g++).	Sin errores.
Parser NMEA del GPS (prueba automatizada).	Correcto (sentencias N/S/E/W y sin fix).
Renderizado del panel (navegador headless).	Sin errores de JavaScript; mapa y tarjetas correctos.
Columna «ip_dispositivo» en Supabase.	Operativa; el ESP32 publica su IP en cada envío.

Prueba pendiente en campo: validar la alarma sonora y el corte del zumbador durante una presentación real con la estación conectada a la red del lugar.

19. Cómo ejecutar y visualizar el sistema

Aplicación web con Live Server (Visual Studio Code)

• Abrir la carpeta App-Web2 en Visual Studio Code.

• Instalar la extensión «Live Server» y pulsar «Go Live».

• Se abrirá el panel en http://127.0.0.1:5500/.

• Desde un celular de la misma red: http://<IP-de-la-PC>:5500/ (la IP se obtiene con hostname -I).

Opción alternativa — servidor local con Python

```
python3 -m http.server 8000 --bind 0.0.0.0
```

• En la misma PC: http://127.0.0.1:8000/

• En un celular de la misma red Wi-Fi: http://<IP-de-la-PC>:8000/

• Requiere conexión a internet (librerías por CDN y consulta a Supabase).

• Para el control del zumbador, estar en la misma red local que el ESP32.

• Detener el servidor con Ctrl + C.

20. Pendientes y recomendaciones

Tema	Recomendación
Publicación online	Hostear los 3 archivos (GitHub Pages, Netlify, Vercel o Cloudflare Pages) para que el público acceda por una URL fija.
Seguridad Supabase	Verificar RLS: lectura pública y bloqueo de escritura salvo el rol del dispositivo.
Realtime	Confirmar que la tabla tenga Realtime habilitado; si no, la app igual funciona por sondeo.
Enlace GitHub	Definir la URL definitiva del repositorio para el botón correspondiente.
Calibración	Calibrar el MQ-7 (R0) y validar el LDR frente a un luxómetro de referencia.
GPS	Usar una antena con buena vista del cielo para mejorar el tiempo de fix.
Alarma	Probar en una presentación real el disparo y el corte remoto del zumbador.
Fuente de alimentación	Usar una fuente capaz de sostener los picos de corriente del Wi-Fi.

Anexo A — Configuración y claves

La configuración se centraliza al inicio de app.js en el bloque CONFIG:

```
const CONFIG = {
supabaseUrl: "https://lskvyvpjjziuclaatrtu.supabase.co",
supabaseKey: "sb_publishable_O9Havbh6tsAZEDA8VFWQzg_LmTmU4mM",
table: "mediciones",
historyLimit: 100,
pollMs: 15000,
staleWarnSeconds: 120,
staleOfflineSeconds: 300,
defaultLat: -26.82414,
defaultLng: -65.2226,
downloadRanges: { "100": 100, "500": 500, "1000": 1000, all: Infinity },
deviceKey: "tucu2026",
deviceIpDefault: "192.168.4.1",
muteMinutes: 10
};
```

Nota de seguridad: «supabaseKey» es una clave pública (anon) pensada para usarse en el cliente; no debe reemplazarse por la clave de servicio. La protección de los datos depende de las políticas RLS del servidor. La clave «deviceKey» es la que exige el ESP32 en el endpoint «/silenciar».

Anexo B — Exportación de datos (CSV / JSON)

CSV

El archivo CSV incluye una marca de orden de bytes (BOM) para correcta apertura en planillas, usa el punto y coma (;) como separador y contiene las columnas:

```
id;fecha_hora;pm25_ug_m3;co2_ppm;co_ppm;temperatura_c;humedad_pct;
presion_hpa;uv;luz_lux;latitud;longitud
```

JSON

El archivo JSON contiene el arreglo de registros tal como los devuelve la API, con todos los campos de cada medición (incluidos latitud, longitud e ip_dispositivo). Se genera sin indentación y con codificación UTF-8 para evitar cortes en archivos grandes.

Anexo C — Trama de comunicación Uno → ESP32

Comunicación serie a 9600 baudios, una línea de texto por medición, con 9 campos separados por comas:

Posición	Campo	Ejemplo
1	CO■ (ppm)	412
2	CO (ppm)	3.20
3	Temperatura (°C)	19.9
4	Humedad (%)	71
5	Presión (hPa)	968
6	UV (índice)	0.0

Posición	Campo	Ejemplo
7	Luz (lux)	120
8	Latitud (grados decimales)	-26.83972
9	Longitud (grados decimales)	-65.29328

El ESP32 procesa la trama con una lectura de formato tipo sscanf("%d,%f,%f,%f,%f,%f,%d,%f,%f") y descarta las tramas inválidas.

Anexo D — Prompt para recrear el proyecto con opencode

A continuación se incluye un prompt ampliado, listo para copiar y pegar en opencode u otro asistente, con la información técnica necesaria para reconstruir el sistema completo: el panel web y los dos firmwares.

```
Actua como desarrollador full-stack senior en sistemas embebidos y frontend. Reconstruye
de forma COMPLETA y FUNCIONAL el sistema "Tucuman Respira": una estacion de monitoreo de calidad
del aire con un Arduino Uno + ESP32 que publica en Supabase, y un panel web estatico que muestra
los datos. Respeta EXACTAMENTE archivos, IDs, textos, unidades, umbrales, colores y
comportamientos
indicados.

=== PANEL WEB (estatico, 3 archivos, sin backend) ===
- index.html (estructura y carga de librerias; script app.js al final)
- styles.css (sistema de diseno con variables de tema)
- app.js (toda la logica; al inicio un bloque CONFIG)
Sin backend propio, sin frameworks de build, sin Node. Funciona sirviendo por HTTP simple.

Organizacion interna de app.js (en orden):
1) "use strict"; y bloque CONFIG.
2) COLOR: mapa nombre->hex de los colores de nivel.
3) METRICS: arreglo con la definicion de los 8 sensores.
4) TREND_TABS: pestanas del grafico de tendencia.
5) AQI_BREAKPOINTS y AQI_CATEGORIES: tablas del indice ICA (EPA).
6) state: objeto unico de estado (rows, latest, trend, modal, modalRange, chart, sensorChart,
map, marker, tile, client, lastId, offlineDismissedId).
7) Utilidades: byId, pad, formatDateTime -> "dd/mm/aaaa hh:mm:ss", relativeTime -> "hace X",
metricById, findRange, formatValue, classify, setLevel, esc, supabaseGet.
8) ICA (EPA): subIndex por interpolacion lineal y computeAQI (peor de pm25 y co) + aqiCategory.
9) Render: buildSkeletons, buildCards, renderCards, renderHero, renderSummary, renderClock,
renderMap, buildTrendTabs, initTrendChart, buildTrendChart, openSensorModal, buildSensorChart,
fetchRows, fetchAllRows, toCSV, download, handleDownload, applyTheme, toggleTheme,
refreshIcons, ingest, renderAll, loadInitial, initRealtime, poll, bindStatic, init.
10) Arranque: document.addEventListener("DOMContentLoaded", init).

Librerias por CDN:
- Iconos: lucide (https://unpkg.com/lucide@latest) -> lucide.createIcons(). OJO: las versiones
nuevas NO incluyen iconos de marcas; el boton de GitHub usa un SVG propio inline.
- Graficos: Chart.js 4.4.1 (cdn.jsdelivr.net/npm/chart.js@4.4.1)
- Mapa: Leaflet 1.9.4 (unpkg.com/leaflet@1.9.4) con mosaicos de OpenStreetMap (SIN clave de
API).
- Realtime: @supabase/supabase-js@2 (cdn.jsdelivr.net)

Fuente de datos (Supabase, solo lectura):
- URL: https://lskvyvpjjziuclaatrtu.supabase.co
- Clave publica (anon): sb_publishable_O9Havbh6tsAZEDA8VFWQzg_LmTmU4mM
- Tabla: mediciones
- Columnas: id, created_at, co2, co, pm25, temperatura, humedad, presion, uv, luz, latitud,
longitud, ip_dispositivo
- GET {URL}/rest/v1/mediciones?select=*&order=id.desc&limit=100
headers: { apikey: <clave>, Authorization: "Bearer <clave>" }
- Realtime: suscripcion a INSERT en public.mediciones; respaldo por polling cada 15000 ms.
```

```
Logica de datos:
- Carga inicial: GET ...&order=id.desc&limit=100 y luego invertir (el mas reciente al final).
- ingest(filas): fusionar por id con un Map, ordenar ascendente y conservar solo las ultimas
500.
- latest = ultimo elemento; lastId = id de latest.
- Realtime INSERT -> ingest([payload.new]) + renderAll().
- Polling cada 15 s: GET ...&limit=1; si cambia la id, ingest + renderAll.
- Un solo state como fuente de verdad.

CONFIG: supabaseUrl, supabaseKey, table="mediciones", historyLimit=100, pollMs=15000,
staleWarnSeconds=120, staleOfflineSeconds=300, defaultLat=-26.82414, defaultLng=-65.2226,
downloadRanges={100:100,500:500,1000:1000,all:Infinity}, deviceKey="tucu2026",
deviceIpDefault="192.168.4.1", muteMinutes=10.

Identidad y tema:
- Nombre: "Tucuman Respira". Fuente sans del sistema. Acento verde esmeralda #10b981 (fuerte
#059669).
- Tema OSCURO: fondo #020617, superficie #0f172a, texto #f1f5f9.
- Tema CLARO: fondo #f3f6fb, superficie #ffffff, texto #0f172a.
- Cambiar con atributo data-theme en <html>; persistir en localStorage; actualizar icono
(sun/moon) y meta theme-color; recolorear graficos y mapa.
- Colores de nivel: emerald #10b981, sky #38bdf8, amber #f59e0b, orange #fb923c, rose #f43f5e,
fuchsia #d946ef, slate #94a3b8. Exponer cada nivel como clase lvl-* que define --level.
- Tokens OSCURO: --bg #020617, --surface #0f172a, --surface-2 #16213b, --border #1e293b,
--border-soft #24314a, --text #f1f5f9, --muted #94a3b8, --muted-2 #64748b.
- Tokens CLARO: --bg #f3f6fb, --surface #ffffff, --surface-2 #f8fafc, --border #e2e8f0,
--border-soft #eef2f7, --text #0f172a, --muted #526072, --muted-2 #7b8798.
- Globales: --accent #10b981, --accent-strong #059669, --radius 18px, --radius-sm 12px.

Secciones de la interfaz:
1) Header fijo: marca + insignia de conexion + botones descargar, tema, info y GitHub.
2) Aviso "offline" descartable con fecha/hora completa de la ultima lectura.
3) Banner ICA: numero grande, categoria, descripcion, recomendacion sanitaria e indicador de
alerta del equipo (NORMAL/PRECAUCION/ALERTA/ALERTA CRITICA).
4) Tarjeta de resumen: temp min/max, pico de PM2.5, promedio de CO2 y humedad.
5) Grilla de 8 tarjetas de sensores (clic -> historico).
6) Panel de tendencia con pestanas: Aire, Clima, Gases, Luz/UV.
7) Panel de mapa (Leaflet/OpenStreetMap) con estado GPS y enlace a Google Maps.
8) Footer centrado.
9) Tres modales: historico por sensor, descarga e informacion.

Metricas, unidades y umbrales (mostrar la FUENTE en la UI):
- CO2 [ppm] OSHA/NIOSH + ASHRAE: Bueno<=800, Aceptable<=1000, Ventilacion pobre<=1500,
Alta (PEL 8h)<=5000, Muy alta (STEL)<=30000, Peligro>30000.
- CO [ppm] EPA (8h)/OSHA: Bueno<=4.4, Moderado<=9.4, Grupos sensibles<=12.4, Danino<=15.4,
Muy danino<=30.4, Peligroso<=50.4, Extremo>50.4.
```

```
- PM2.5 [ug/m3] EPA AQI 2024: Bueno<=9, Moderado<=35.4, Danino grupos sensibles<=55.4,
Danino<=125.4, Muy danino<=225.4, Peligroso>225.4.
- Temperatura [C] NWS/OSHA (calor) + NIOSH (frio): Hipotermia<=0, Frio<=10, Confortable<=27,
Precauccion por calor<=32, Extrema precaucion<=39, Peligro<=52, Peligro extremo>52.
- Humedad [%] OMS + ASHRAE: Muy seco<=20, Seco<=30, Ideal<=60, Algo humedo<=70,
Riesgo de moho<=85, Alto riesgo de moho>85.
- Presion [hPa] Informativo: Baja<=1004.9, Normal<=1020, Alta>1020.
- UV [indice] OMS/ICNIRP: Bajo<=2, Moderado<=5, Alto<=7, Muy alto<=10, Extremo>10.
- Luz [lux] Informativo: Oscuridad<=9.9, Penumbra<=99.9, Interior<=499, Pleno sol>499.

ICA (EPA): subindice de PM2.5 y CO por interpolacion lineal; tomar el PEOR.
- pm25: [0,9.0]->[0,50] [9.1,35.4]->[51,100] [35.5,55.4]->[101,150] [55.5,125.4]->[151,200]
[125.5,225.4]->[201,300] [225.5,500.4]->[301,500]
- co: [0,4.4]->[0,50] [4.5,9.4]->[51,100] [9.5,12.4]->[101,150] [12.5,15.4]->[151,200]
[15.5,30.4]->[201,300] [30.5,40.4]->[301,400] [40.5,50.4]->[401,500]
- Categorias: 0-50 Buena; 51-100 Moderada; 101-150 Danina para grupos sensibles; 151-200
Danina; 201-300 Muy danina; 301-500 Peligrosa.

Mapa: Leaflet con mosaicos de OpenStreetMap (sin clave). Marcador si hay latitud/longitud
validas;
si la lectura actual no tiene posicion, usar la ULTIMA coordenada valida y mostrar "Ultima
ubicacion conocida". invalidateSize() al redimensionar u orientar.

Descarga: modal con formato CSV o JSON; rango 100 (defecto)/500/1000/todas. CSV con BOM UTF-8 y
separador ";", columnas: id;fecha_hora;pm25_ug_m3;co2_ppm;co_ppm;temperatura_c;humedad_pct;
presion_hpa;uv;luz_lux;latitud;longitud. "Todas" via paginacion con header Range. JSON sin
indentacion y con MIME application/json;charset=utf-8. Nombre
tucuman_respira_AAAA-MM-DD.csv/.json.

Boton "Detener zumbador": obtiene la IP del dispositivo desde la columna ip_dispositivo de
Supabase (fallback 192.168.4.1) y llama a http://<ip>/silenciar?k=tucu2026.

Modales/accesibilidad: openModal/closeModal con bloqueo del scroll; cerrar con X, fondo y
Escape;
role="status" aria-live="polite" en la insignia; role="dialog" aria-modal en modales; aria-label
en botones de icono; respetar prefers-reduced-motion. Registrar la regla CSS
[hidden]{display:none !important} (imprescindible: .modal usa display:grid/flex).

=== FIRMWARE ARDUINO UNO (sketch_oct17a) ===
- BME280 (I2C A4/A5), MH-Z19C (SoftwareSerial RX=10 TX=11), MQ-7 (A0), modulo LDR (A1/D12),
GY-8511 ML8511 (A2), GPS NEO-6M en SoftwareSerial RX=7 TX=8, ESP32 por SoftwareSerial RX=2 TX=3.
- Interpreta $GPGGA y $GNGGA: verifica el fix y convierte lat/lon de NMEA (ggmm.mmmm) a grados
decimales (signo segun N/S y E/W). Si pierde el fix, NO envia (0,0): conserva la ultima
coordenada valida.
- MQ-7 con calentamiento inicial de 30 s. Proteccion contra NaN del BME280.
- Cada 3 s envia al ESP32 una trama de 9 campos: co2,co,temperatura,humedad,presion,uv,luz,
latitud,longitud.
```

```
=== FIRMWARE ESP32 (sketch_oct17b) ===
- WiFi en modo estacion (STA) con reconexion automatica + punto de acceso permanente
"Tucuman-Respira-Config" (clave Tucuman2026, IP 192.168.4.1) con portal DNS cautivo; guardar
credenciales en NVS con Preferences.
- POST HTTPS a Supabase con headers apikey/Authorization. Campos: co2, co, pm25, temperatura,
humedad, presion, uv, luz, latitud, longitud, ip_dispositivo (= WiFi.localIP()).
- Zumbador pasivo en GPIO 23 (tone/noTone). ICA local con breakpoints EPA 2024. Alarma on si
ICA>100,
off si <80; silencio manual 10 minutos.
- Endpoints: /silenciar?k=tucu2026, /status, /alarma, /save (POST), y portal "/" de
configuracion
que confirma el estado del WiFi. CORS abierto (Access-Control-Allow-Origin: *).
- Sensor de particulas PMS5003 por Serial2 (RX 18 / TX 5, SET 4).

=== SUPABASE ===
- Tabla mediciones: id, created_at, co2, co, pm25, temperatura, humedad, presion, uv, luz,
latitud, longitud, ip_dispositivo.
- Agregar la columna: alter table public.mediciones add column if not exists ip_dispositivo
text;

=== DATOS DE LA INTERFAZ (textos) ===
- Marca: "Tucuman Respira" / "Monitoreo ambiental IoT".
- Insignia: "Conectando...", "En vivo · <rel>", "Retraso · <rel>", "Sin conexion · <rel>".
- Aviso offline: "La estacion no ha enviado lecturas desde <fecha> (<rel>). Mostrando los
ultimos
datos registrados."
- Footer: "Tucuman Respira · Estacion de monitoreo ambiental IoT".

=== RESPONSIVE ===
- <=1024px: hero a 1 columna; sensores a 2; tendencia y mapa apilados.
- <=640px: header compacto; sensores a 1 columna; graficos/mapa 240-260px; modales con menos
margen.
- <=400px: aun mas compacto. overflow-x oculto en html/body.

=== CRITERIOS DE ACEPTACION ===
1) Los 3 archivos cargan sin errores en consola.
2) Registrar [hidden]{display:none !important}.
3) Las 8 tarjetas se pintan; campos nulos muestran "--" solo en su metrica.
4) El ICA es el peor subindice entre PM2.5 y CO.
5) Tendencia e historico se dibujan sin colapsar en escritorio y movil.
6) Descargas CSV/JSON validas; "Todas" trae todo.
7) El mapa muestra marcador si hay coordenadas; si no, usa la ultima valida o "Sin senal".
8) El aviso offline aparece tras 300 s y es descartable.
9) Sin scroll horizontal a 360 px.
10) El tema persiste al recargar y recolorea graficos y mapa.
11) Ambos firmwares compilan (avr-g++ y xtensa-esp32-elf-g++) sin errores.

Al terminar, entrega los archivos completos, sin marcadores de posicion (TODO) ni codigo
incompleto, listos para usar.
```

Fin del informe — «Tucumán Respira», sistema de monitoreo de calidad del aire IoT (panel web, umbrales y firmwares). Encargado del trabajo: **Benicio Gael Pieroni**. Documento generado el 18/10/2026.